Introduction to Software Engineering and Models

- Foundational Concepts
  - The evolving role of software
  - Changing nature of software
  - Software myths
- Software Engineering Principles
  - Software engineering as a layered technology
  - A process framework
  - Process patterns
  - Process assessment
  - Personal and team access models
- Specific Process Models
  - The waterfall model
  - Incremental process models
  - Evolutionary process models
  - The unified process

Software

A software is basically a collection of programs, instructions, and data that tells a computer or electronic device how to perform specific tasks.

The Evolving Role of Software

- This refers to how software has transformed from simple programs in the mid-20th century (like basic calculators or payroll systems) to the backbone of the modern society.
- As software role's expands, it impacts economies, healthcare, entertainment, and more. For instance, during the Covid-19 pandemic, software allowed people to work remotely via tools like Zoom, Google Meet, etc. Understanding this evolution helps engineers anticipate future needs, like ethical AI, sustainable computing, etc.
- What it means for developers is that they must adapt to the new technologies by learning new paradigms, like shifting from desktop software to web-based services.
- **Example**: When Instagram first launched, it was just a simple photo sharing app but over the years it has evolved into a social commerce platform.

The Changing Nature of Software

- Software development has become more complex due to factors like larger scale (e.g.: billions of users for Facebook, Instagram, Whatsapp, etc.), integration with hardware (IoT, sensors, etc.) and rapid tech changes (monolithic to microservices).
- This shift means that traditional ad-hoc coding is no longer suitable and requires disciplined approach to engineering to tackle risks like bugs, breaches, or scalability issues.

- Developers now deal with distributed systems (software running across multiple different systems) where requirements of the projects change mid-session. Tools like `git` help keep track of these changes.

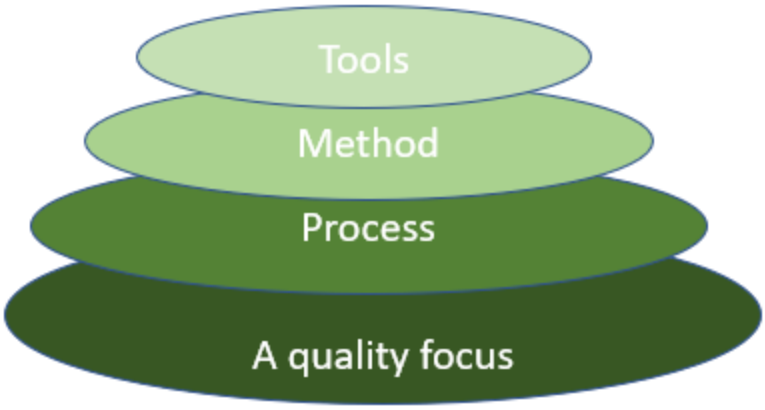
Software Myths

- Software myths are widespread misconceptions about software and its development process such as "Software is easy to build and ship" or "Adding more programmers will speed up the development of an app" whereas in reality it slows things down due to communication overhead.
- Treat software engineering like any other engineering field-systematic and evidence based.

Software Engineering Principles

Software Engineering as Layered Technology

Software Engineering is a fully layered technology, to develop software we need to go from one layer to another. All layers are connected and each layer demands fulfilment of the previous layer.



1. **A quality focus**
    - Ensures the continuous improvement cycle for the software's development process.
    - It also provides integrity and security to the software so that only authorized persons are able to access it and make any changes to it.
    - It also focuses on maintaining readability and usability of the software.
2. **Process**
    - It is the foundation or base layer of software engineering.
    - It is the key that binds all the layers together and enables the development of software on time.
    - Process defines a framework that must be established for the effective delivery of the software.

- Process activities are:
    1. Communication
        - It is the first and foremost thing for the development of software.
        - Communication is necessary to know the actual requirements of the client.
    2. Planning
        - It is the foundational phase that defines a project's goals, requirements, scope and resources to guide the development process and ensure successful delivery.
    3. Modelling
        - This includes making abstract layers of the software system to visualize, analyze, and refine its structure, behavior, and interactions.
        - It's not about writing code yet; instead, it's about making diagrams, notations, charts so that it's easy to understand by stakeholders (like developers, designers, clients).
    4. Construction
        - This is the layer that involves hands-on development of the software.

        - However, the sole focus is not entirely on coding---it's a structured process that emphasizes efficiency, error prevention, and adherence to best practices.
    5. Deployment
        - It includes delivery of the software to the client for evaluation and feedback.

3. **Method**
    - This method layer encompasses the standardized approaches, tools, and best practices used to execute software engineering activities.
    - It's not a sequential layer like `modelling` or `construction`; instead, it's an overarching layer which influences every stage of the process.
4. **Tools**
    - The tools layers consist of technological enablers--software applications, IDEs, platforms and supporting infrastructures that automate, support, and enhance the software engineering activities across all the other layers.
    - It's not about creating the software itself but about providing the means to do it effectively.

Process Framework

A process framework is a high level, generic template or skeleton that outlines the essential activities, workflows, and milestones for software development.

> It's like a customizable blueprint for the entire software engineering process, providing a foundation that can be adapted to specific projects or methodologies (e.g.: waterfall, agile, or spiral models)

Need for Process Framework:

1. It promotes consistency and continuous process improvement by standardizing the "what" and "when" of development, reducing chaos and errors.
2. Without it, teams might re-invent the wheel, leading to inefficiencies or security gaps.
3. It directly supports maintainability by ensuring structured handoffs between layers.

Process Patterns

Process patterns are reusable, proven solutions to common challenges in software development processes, similar to design patterns in coding but applied at the process level.

> They describe the best practices for handling recurring problems, developed from real world experiences, they can be combined like building blocks to customize a process framework.

Need for Process Patterns:

1. They enable continuous improvements by distilling lessons from successful projects, thus helping prevent pitfalls like poor maintainability or integrating breaches.

2. Learning from patterns observed in multiple projects helps teams evolve without starting from scratch, reducing project failures.

Process Assessment

Process Assessment is the systematic evaluation of a software development process to determine its maturity, effectiveness, strengths, and areas for improvements.

> It involves audits, metric analysis, and comparisons against standards to gauge how well the process delivers quality software.

Need for Process Assessment:

1. It drives continuous process improvement by identifying gaps, such as weak security practices that could allow unauthorized data access, or inflexible designs that hinder maintainability.

Personal and Team Access Models

pata nahi kya hota hai

Specific Process Models

In software development, a "process model" is a roadmap or recipe for creating software. It outlines the phases, activities, and order in which things happen to turn an idea into a working product.

Waterfall Model

- The waterfall model is one of the oldest and most straightforward software development processes, introduced by Winston Royce in 1970.
- It's called waterfall because the phases flow sequentially downward, and you can't go back up easily.

How it works?

1. **Requirements Gathering and Analysis**: Collect and document all user needs upfront. This is like creating a detailed shopping list before starting a recipe-no changes allowed later.
2. **System Design**: Based on the requirements, create a high-level (overall architecture) and low-level (detailed components) designs. This includes deciding on databases, interfaces, etc.
3. **Implementation (Coding)**: Developers write the actual code, turning designs into executable software.
4. **Verification (Testing)**: Test the entire system for bugs, performance issues, and correctness. This includes unit tests, integration tests, and user acceptance tests.
5. **Deployment**: Release the software to the users.
6. **Maintenance**: Keep track of issues post-release and actively fix them and add new features.

> The key principle in linearity. Each phase must be fully completed and approved before moving to the next.

Pros:

1. Simple and easy to manage-clear milestone and deliverables make it easy for budgeting and scheduling.
2. Strong documentation helps in regulated industries (e.g.: healthcare and finance).
3. Predictable timeline, like a conveyor belt in a factory.

Cons

1. Inflexible: If requirements changes midway, you have to restart.
2. Late testing means bugs are discovered late, increasing costs (fixing a bug in maintenance can cost 100x more than in design).
3. Not ideal for complex, evolving projects like web apps where user feedback is crucial.

Incremental Process Models

- Incremental Process Models build on Waterfall models but break the project into smaller, manageable "increments" or chunks.
- Instead of delivering the whole product at once, you deliver functional pieces iteratively, adding more over time.

How it works?

1. **Initial Planning**: Define the overall requirements and prioritize features into increments (e.g.: Increment 1: Core login system; Increment 2: User profiles).
2. **For Each Increment**:
    - Requirements for that chunk only.
    - Design, Implement, test, and deploy just that part (mini-Waterfall cycle).
    - Integrate it with previous increments.
3. **Iteration and Feedback**: After each delivery, gather use input to refine future increments.
4. **Final Integration**: Once all increments are done, the full system is complete, followed by maintenance.

> The process is still somewhat sequential per increment but allows for evolution across them. Tools like version control
> (e.g.: Git) help manage this.

> Software projects have evolving needs, so this model uses inductive reasoning: From small successes (each increment), you build towards the whole.

Pros:

1. Early delivery of usable software provides quick wins and stakeholders buy-in.
2. Easier to manage changes---adjust future increments without scraping everything.
3. Cost-effective for budget constrained teams, as you can stop after a viable product.

Cons:

1. Requires good planning to define increments; poor prioritization can lead to uneven quality.
2. Integration issues between increments.
3. Still somewhat rigid within each increment, so not as flexible as fully iterative models.

Evolutionary Process Models

- Evolutionary models emphasize building prototypes and iterating based on feedback, allowing the software to "evolve" over time like a living organism adapting to its surrounding.
- It's a response to the fact that requirements are often unclear initially---why build the full thing if you don't know what the users want?

How it works?

1. **Initial Prototype**: Quickly build a basic version (throwaway or evolutionary prototype) to demonstrate core ideas.
2. **Feedback Loop**: Users test it, provide input, and the team refines (e.g.: add features, fix flaws)
3. **Iterations**: Repeat prototyping, evaluation, and refinement in cycles until the product meets needs.
4. **Final Delivery**: Converge on a stable version for deployment and maintenance.

> Using abductive reasoning (best guess based on incomplete info), it assumes software needs to evolve, so constant adaptation is key.

Pros:

1. Highly Flexible---handles uncertainty and changes well, ideal for innovative products.
2. User involvement leads to higher satisfaction and fewer surprises.
3. Risk mitigation: Identify issues early through prototypes.

Cons:

1. It can lead to "scope creep" if iterations never end (endless tweaking).
2. Resource-intensive: Requires skilled teams for rapid prototyping and may produce incomplete products initially.
3. Hard to estimate costs or timeline due to iterative nature.

The Unified Process

- It is a comprehensive, iterative framework that's use-case driven, architecture-centric, and risk-focused.
- It unifies various best practices from earlier models (e.g.: iterative like Evolutionary, but structured).

How it works?

1. **Four phases**
    - **Inception**: Define scope, business care, and high-level risks
    - **Elaboration**: Build architecture, resolve major risks, and create detailed use cases
    - **Construction**: Develop the bulk of code iteratively, in small releases
    - **Transition**: Deploy, train users, and gather feedback for beta testing.
2. **Disciplines**: Run in parallel---requirements, analysis/design, implementation, testing, deployment, etc.
3. **Iterations**: Within phases, work in short cycles (e.g.: 2-6 weeks) with milestones.
4. **Artifacts**: Emphasizes models like UML (Unified Modelling Language) for visualization.

Pros:

1. Balanced and scalable---iterative yet disciplined, great for large, complex systems.
2. Strong on quality: Built-in testing and risk management.
3. Promotes reuse (e.g.: components from past projects)

Cons:

1. Steep learning curve---requires training in tools like UML

2. Can be bureaucratic with too much documentation for small teams.
3. Overkill for simple apps, leading to unnecessary overhead.
